



utbm

université de technologie
Belfort-Montbéliard

Projet AC20 - Classification d'Images (Chien vs Chat) avec CNN

Université de Technologie de Belfort-Montbéliard (UTBM)

Unité de Valeur : AC20

Étude et Implémentation de Réseaux de Neurones Convolutifs (CNN)

Problématique : Comment construire, entraîner et optimiser un réseau de neurones convolutif (CNN) à partir de zéro, et quel est l'apport réel du Transfer Learning par rapport à une architecture artisanale sur un problème de classification binaire ?

Enseignant Responsable : M. Abdeljalil ABBAS-TURKI

Élève : Abd-Allah AMINI

Sommaire

1. Avant-propos
2. Introduction
3. Étude Bibliographique et Théorie
 - 3.1 Contextualisation
 - 3.2 L'inspiration biologique
 - 3.3 Modélisation mathématique
 - 3.4 Le Perceptron Multicouches (MLP)
 - 3.5 Les réseaux de neurones convolutifs (CNN)
4. Développement et Implémentation
 - 4.1. Configuration de l'Environnement de Travail
 - 4.2. Préparation des Données et Analyse Exploratoire (EDA)
 - 4.3. Expérience 1 : Le Modèle Baseline ("From Scratch")
 - 4.4. Expérience 2 : Optimisation et Régularisation
 - 4.5. Analyse Comparative : Optimisé VS Baseline
 - 4.6. Expérience 3 : Approche par Transfer Learning (Resnet50)
5. Analyse Globale des Résultats
 - 5.1. Synthèse Métrique et Matrice Multicritères
 - 5.2. Analyse Comparative des Comportements Inductifs
 - 5.3. Analyse Technique des Erreurs et Limites de Généralisation
 - 5.4. Arbitrage Industriel : Le Compromis Performance-Coût
6. Conclusion
7. Bibliographie et Annexes

1. Avant-propos

Ce présent rapport marque l'aboutissement du projet mené dans le cadre de l'Unité de Valeur AC20, au sein de l'Université de Technologie de Belfort-Montbéliard. Représentant un investissement estimé à 150 heures de travail, cette étude portant sur la classification d'images par réseaux de neurones convolutifs (CNN) a constitué un défi technique et scientifique majeur.

Au-delà de la simple validation d'acquis académiques, ce travail s'inscrit comme une étape décisive dans mon projet professionnel visant à devenir Data Scientist. Il m'a offert l'opportunité de me confronter aux contraintes réelles de l'ingénierie des données et de l'apprentissage automatique : de la conception d'une architecture algorithmique robuste jusqu'à l'analyse critique des performances face au risque de sur-apprentissage, en passant par le déploiement applicatif.

Je tiens à adresser mes sincères remerciements à Monsieur Abdeljalil ABBAS-TURKI pour son encadrement pédagogique, sa disponibilité et ses conseils avisés tout au long de ce semestre.

Enfin, la réalisation d'un projet d'une telle exigence nécessite un équilibre et un environnement personnel solides. Je souhaite donc remercier chaleureusement mon épouse pour sa patience et son soutien indéfectible au cours de ces nombreuses heures de développement, un soutien d'autant plus précieux au quotidien depuis l'arrivée de notre fille.

2. Introduction

Au cours de la dernière décennie, la vision par ordinateur a connu un bouleversement paradigmatique grâce à l'essor de l'apprentissage profond (*Deep Learning*). Là où les approches statistiques traditionnelles peinaient à extraire manuellement des caractéristiques pertinentes face à la variabilité infinie du monde réel (changements d'échelle, d'éclairage, occultations ou variations de pose), les architectures profondes ont démontré une capacité inédite à modéliser des représentations complexes directement à partir des pixels bruts. Au cœur de cette révolution algorithmique se trouvent les Réseaux de Neurones Convolutifs (CNN), qui exploitent astucieusement la topologie spatiale des images pour s'imposer comme le standard incontesté de l'analyse visuelle contemporaine.

Ce projet s'inscrit pleinement dans cette dynamique technologique en s'attaquant à un cas d'étude fondateur : la classification binaire d'images de chiens et de chats.

Cependant, au-delà de la simple résolution d'un problème de classification canonique, ce travail s'articule comme une véritable démarche d'ingénierie des données. L'enjeu est de dépasser l'utilisation de modèles "boîtes noires" pour maîtriser l'intégralité du cycle de vie d'un algorithme d'apprentissage supervisé. De la structuration d'un pipeline de données robuste jusqu'au déploiement et à l'évaluation critique du modèle final, cette démarche reflète les exigences de reproductibilité et de performance indispensables dans la conception de systèmes d'intelligence artificielle modernes.

C'est dans ce cadre d'exigence scientifique que s'inscrit la problématique centrale de ce rapport : **Comment construire, entraîner et optimiser un réseau de neurones convolutif à partir de zéro, et quel est l'apport réel du Transfer Learning par rapport à une architecture artisanale sur un problème de classification binaire ?**

Pour y répondre, une méthodologie expérimentale et incrémentale a été déployée. La stratégie consiste d'abord à prouver la viabilité d'une architecture conçue sur mesure (*Baseline*), afin de se confronter aux défis inhérents à l'entraînement initial. Une fois les faiblesses identifiées — et plus particulièrement la forte propension au sur-apprentissage face à des données complexes — des stratégies de régularisation avancées (telles que la *Data Augmentation*, le *Dropout* et la *Batch Normalization*) seront intégrées pour stabiliser le réseau. Enfin, les limites computationnelles de cette approche purement artisanale seront éprouvées par une mise en concurrence directe avec l'état de l'art, via l'exploitation d'un modèle pré-entraîné (ResNet50) adaptant ses poids par *Transfer Learning*.

Afin de retracer rigoureusement cette démarche, le présent document est structuré en quatre axes principaux :

- **Dans un premier temps**, l'étude bibliographique (Section 3) posera les fondements théoriques, justifiant mathématiquement le passage du neurone formel aux architectures convolutives pour le traitement d'images.
- **Dans un second temps**, la phase de préparation (Sections 4.1 et 4.2) exposera la configuration de l'environnement matériel accéléré par GPU, ainsi que l'analyse exploratoire (EDA) et la standardisation stricte du jeu de données.
- **La troisième partie** (Sections 4.3 et 4.4) constituera le cœur du développement itératif, en détaillant l'implémentation de la *Baseline* logicielle suivie de son optimisation architecturale.
- **Pour conclure**, la dernière section (Section 4.5 et 5) abordera l'intégration du *Transfer Learning* (ResNet50) et synthétisera l'ensemble de ces expériences dans une analyse comparative globale évaluant la robustesse et les performances de chaque modèle. Enfin, en guise d'ouverture technologique, le rapport présentera un test exploratoire de l'architecture YOLO (*You Only Look Once*), marquant ainsi la transition conceptuelle d'un problème de classification pure vers les enjeux de la détection d'objets en temps réel.

3. Étude Bibliographique et Théorie

3.1. Contextualisation

L'intelligence artificielle (IA) englobe l'ensemble des techniques visant à conférer à une machine des capacités d'analyse et de décision. Au sein de ce spectre, l'apprentissage automatique (*Machine Learning*) a opéré un changement de paradigme fondamental : plutôt que de programmer explicitement des règles métiers déterministes, l'ingénieur conçoit un algorithme capable d'ajuster ses propres paramètres statistiques en extrayant des motifs récurrents à partir d'un jeu de données.

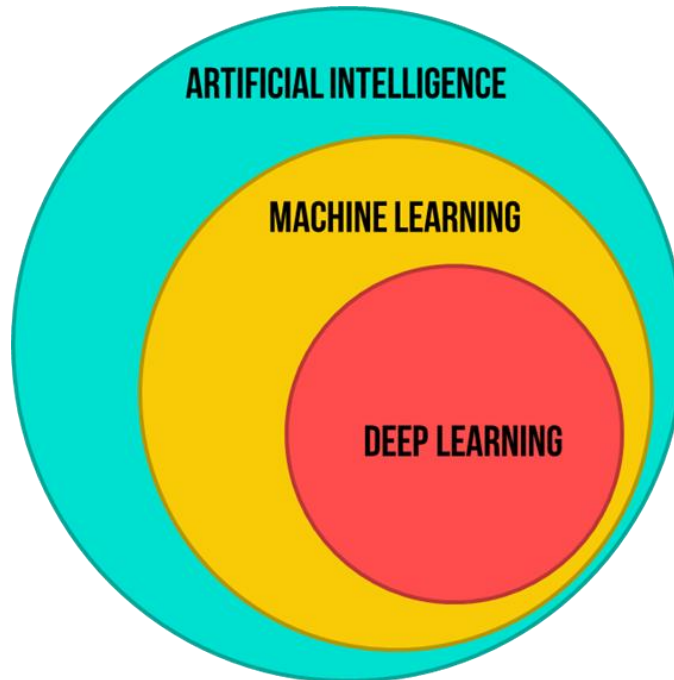


Figure 1 : Les différents domaines de l'IA

Cependant, les modèles de Machine Learning classiques (tels que les Séparateurs à Vaste Marge ou les Random Forest) présentent une vulnérabilité structurelle face aux données non structurées, comme les images. Leur performance dépend presque intégralement de la qualité du *Feature Engineering*.

Historiquement, pour classer une image, il était nécessaire de concevoir manuellement des filtres mathématiques (comme les filtres de Gabor ou les descripteurs HOG) pour extraire les contours, les textures ou les gradients de luminosité avant de les fournir au classifieur.

Cette extraction manuelle de caractéristiques est coûteuse, spécifique à chaque problème, et montre rapidement ses limites face à la variabilité du monde réel (changements d'échelle, d'éclairage ou d'occlusion).

Le *Deep Learning* apporte une réponse structurelle à cette limitation en introduisant l'apprentissage de représentations (*Representation Learning*). En s'appuyant sur des architectures composées de multiples strates de traitement (les couches cachées), un modèle d'apprentissage profond n'a besoin que des données brutes en entrée.

Le réseau apprend de manière autonome et hiérarchique à extraire des descripteurs pertinents : les couches initiales identifient des primitives géométriques de bas niveau (bords, angles), tandis que les couches plus profondes combinent ces primitives pour

modéliser des concepts sémantiques de haut niveau (des oreilles, un musée, puis l'entité globale "chien" ou "chat").

C'est cette abstraction itérative et automatisée qui justifie l'utilisation exclusive du Deep Learning pour les tâches de vision par ordinateur contemporaines.

3.2. L'inspiration biologique

Historiquement, la structure de ces architectures en couches tire son origine d'une tentative de modélisation computationnelle du système nerveux central. Le neurone biologique est une cellule asymétrique constituée de dendrites (qui collectent les signaux électriques d'autres neurones), d'un corps cellulaire et d'un axone (qui achemine le potentiel d'action généré vers d'autres synapses, sous réserve qu'un seuil d'excitation ait été franchi).

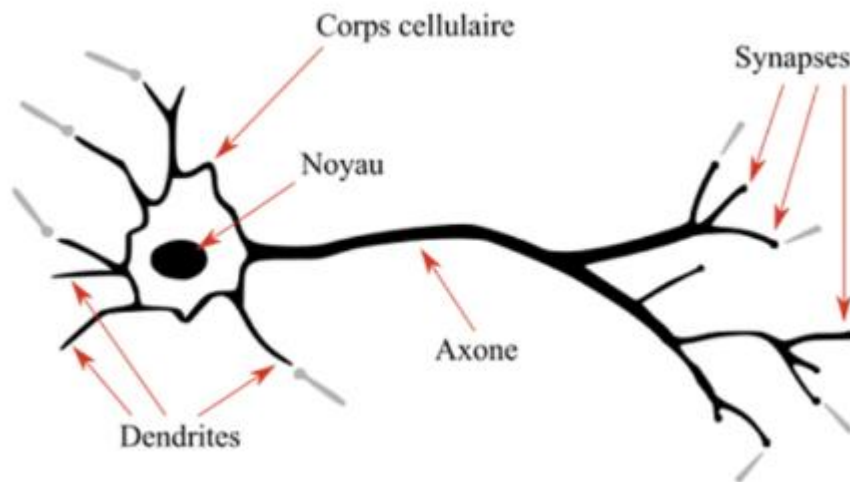


Figure 2 : représentation d'un neurone biologique

Selon le postulat de Hebb (1949), l'apprentissage biologique repose sur la plasticité synaptique, c'est-à-dire le renforcement ou l'affaiblissement de l'efficacité de la transmission chimique entre deux neurones simultanément actifs.

C'est cette mécanique d'intégration à seuil et de pondération synaptique qui a inspiré le modèle fondateur du **neurone formel** (McCulloch et Pitts, 1943) puis du Perceptron (Rosenblatt, 1957).

Il est toutefois impératif, dans une démarche d'ingénierie rigoureuse, de s'affranchir de l'analogie biologique pour aborder ces modèles. Le neurone artificiel contemporain ne simule en rien la complexité biochimique, spatiale et temporelle d'un réseau neuronal vivant. Il s'agit d'une abstraction mathématique drastique, conçue pour être différentiable et optimisable par des méthodes d'analyse numérique sur des architectures matérielles de type GPU.

Plutôt que de voir le Deep Learning comme un "cerveau artificiel", il convient de le définir techniquement pour ce qu'il est : un algorithme d'approximation universelle de fonctions non-linéaires, dont l'apprentissage repose sur l'optimisation stochastique d'un espace de paramètres de grande dimension.

3.3. Modélisation mathématique du neurone artificiel

D'un point de vue topologique et algébrique, un neurone artificiel — ou perceptron de Rosenblatt — est un opérateur mathématique qui réalise une application non-linéaire d'un espace vectoriel d'entrée de dimension n vers un espace de sortie généralement unidimensionnel.

Le traitement de l'information au sein d'un neurone se décompose en une combinaison affine suivie d'une projection non-linéaire.

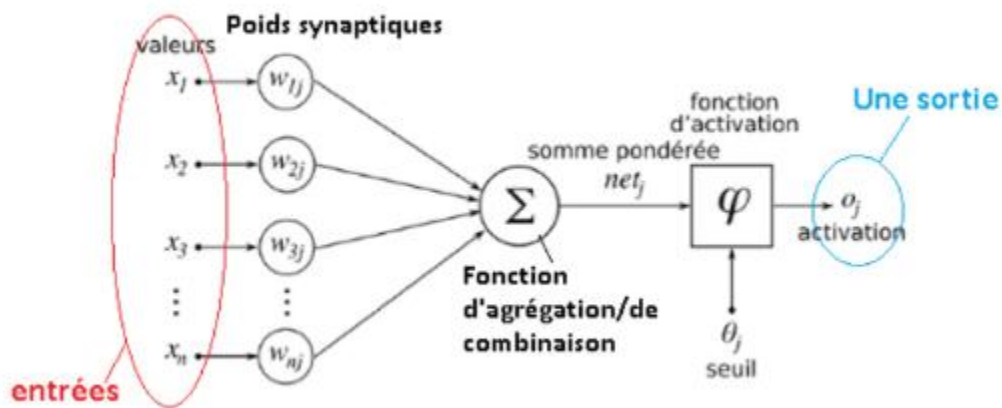


Figure 3 : représentation d'un neurone artificiel

3.3.1. Formalisation de la combinaison affine

Soit un vecteur d'entrée $\mathbf{x} \in \mathbb{R}^n$, où chaque composante x_i représente une caractéristique quantitative (dans le cas de la vision par ordinateur, l'intensité normalisée d'un pixel ou la valeur d'une carte de caractéristiques).

À ce vecteur est associé un vecteur de **poids synaptiques** $\mathbf{w} \in \mathbb{R}^n$, dont chaque composante w_i quantifie l'amplitude de la connexion ou l'importance de la caractéristique x_i vis-à-vis de la décision du neurone.

La première étape consiste à calculer le potentiel d'activation net, noté $z \in \mathbb{R}$. Mathématiquement, z est le produit scalaire (ou forme bilinéaire symétrique) du vecteur des poids et du vecteur d'entrée, auquel est adjoint un paramètre de translation nommé **biais** ($b \in \mathbb{R}$) :

$$z = \langle \mathbf{w}, \mathbf{x} \rangle + b = \sum_{i=1}^n w_i x_i + b$$

En notation matricielle, en considérant $\mathbf{w}$ et $\mathbf{x}$ comme des vecteurs colonnes, cette relation s'écrit sous forme d'une transformation affine :

$$z = \mathbf{w}^T \mathbf{x} + b$$

Interprétation géométrique : L'équation $\mathbf{w}^T \mathbf{x} + b = 0$ définit un **hyperplan de décision** de dimension $n - 1$ dans l'espace $\mathbb{R}^n$. Le vecteur $\mathbf{w}$ est orthogonal à cet hyperplan, délimitant l'espace en deux demi-espaces. La valeur de z mesure la distance signée d'un point de donnée $\mathbf{x}$ à cet hyperplan.

3.3.2. L'opérateur d'activation non-linéaire

Le potentiel net z est ensuite soumis à une fonction d'activation $f: \mathbb{R} \rightarrow \mathbb{R}$. Sans cette fonction, toute composition de neurones (réseaux multicouches) se réduirait à une succession de transformations affines, dont la composition reste affine, restreignant le modèle à la résolution de problèmes linéairement séparables.

Le choix de f et la connaissance de sa dérivée f' sont fondamentaux pour l'algorithme de rétropropagation du gradient.

1. La fonction Rectified Linear Unit (ReLU)

Utilisée préférentiellement dans les couches cachées des CNN (et notamment dans l'architecture Baseline de ce projet), elle est définie par l'opérateur de maximisation :

$$f(z) = \max(0, z)$$

Sa dérivée (ou sous-gradient en $z=0$) est une fonction marche d'Heaviside :

$$f'(z) = \frac{df(z)}{dz} = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z < 0 \end{cases}$$

Avantage mathématique : Contrairement aux fonctions saturantes, sa dérivée est constante et égale à 1 pour toutes les valeurs positives. Cela permet d'annuler le phénomène de disparition du gradient (*vanishing gradient*) sur les réseaux profonds, garantissant un flux constant du gradient lors de la rétropropagation.

2. La fonction Logistique (Sigmoid)

Historiquement cruciale, elle réalise une bijection de $\mathbb{R}$ vers l'intervalle ouvert $]0,1[$, ce qui permet de l'interpréter comme une probabilité a posteriori :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sa propriété remarquable réside dans la simplification algébrique de sa dérivée, qui s'exprime directement en fonction de sa propre sortie :

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$

Inconvénient : Lorsque $|z|$ devient grand, $\sigma'(z)$ tend asymptotiquement vers 0. Ce blocage des gradients ralentit, voire stoppe l'apprentissage dans les couches inférieures d'un réseau profond.

3.3.3. Vectorisation à l'échelle d'une couche et d'un batch

Dans une architecture de Deep Learning, les neurones ne sont pas isolés mais regroupés par couches. Soit une couche de m neurones recevant un vecteur d'entrée $\mathbf{x} \in \mathbb{R}^n$. Les vecteurs de poids individuels $\mathbf{w}_j$ (pour $j = 1, \dots, m$) sont regroupés dans une matrice de poids $\mathbf{W} \in \mathbb{R}^{m \times n}$ et les biais dans un vecteur $\mathbf{b} \in \mathbb{R}^m$.

La transformation de la couche s'écrit :

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{y} = f(\mathbf{z})$$

Où f est appliquée élément par élément (*element-wise*).

Pour optimiser les calculs via le parallélisme des architectures GPU (comme la RTX 3050 utilisée ici), on traite simultanément un ensemble de B exemples (le *Batch Size*). Soit $\mathbf{X} \in \mathbb{R}^{B \times n}$ la matrice d'entrée. L'équation se généralise sous la forme :

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T + \mathbf{1}_B\mathbf{b}^T$$

Où $\mathbf{1}_B \in \mathbb{R}^B$ est un vecteur colonne de uns permettant d'étendre par diffusion (*broadcasting*) le vecteur de biais à chaque ligne du batch.

2.3.4. Rétropropagation et formulation de l'erreur

L'apprentissage consiste à minimiser une fonction de coût globale $\mathcal{L}$. Pour mettre à jour un poids spécifique w_{ij} reliant l'entrée i au neurone j , on applique le théorème de dérivation des fonctions composées (*Chain Rule*) afin d'obtenir la dérivée partielle de la perte par rapport à ce poids :

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial y_j} \cdot \frac{\partial y_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

En exploitant les définitions précédentes, nous savons que

$$\frac{\partial z_j}{\partial w_{ij}} = x_i \text{ et } \frac{\partial y_j}{\partial z_j} = f'(z_j).$$

Par conséquent :

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial y_j} \cdot f'(z_j) \cdot x_i$$

Cette formulation mathématique montre explicitement l'impact direct de la fonction d'activation (f') et de la valeur d'entrée (x_i) sur l'amplitude de la correction du paramètre, formalisant ainsi la dynamique d'apprentissage du neurone.

3.4. Le Perceptron Multicouche (MLP)

3.4.1. Architecture Globale et Propagation Avant

Le Perceptron Multicouche (MLP — *Multi-Layer Perceptron*) représente l'architecture fondatrice des réseaux de neurones artificiels profonds. Il est constitué d'une couche d'entrée, d'une ou plusieurs couches intermédiaires dites « cachées », et d'une couche de sortie. La caractéristique structurelle essentielle du MLP est qu'il s'agit d'un réseau entièrement connecté (*Fully Connected*) : chaque unité de calcul (neurone) d'une couche l reçoit en entrée les sorties de l'intégralité des neurones de la couche précédente $l - 1$.

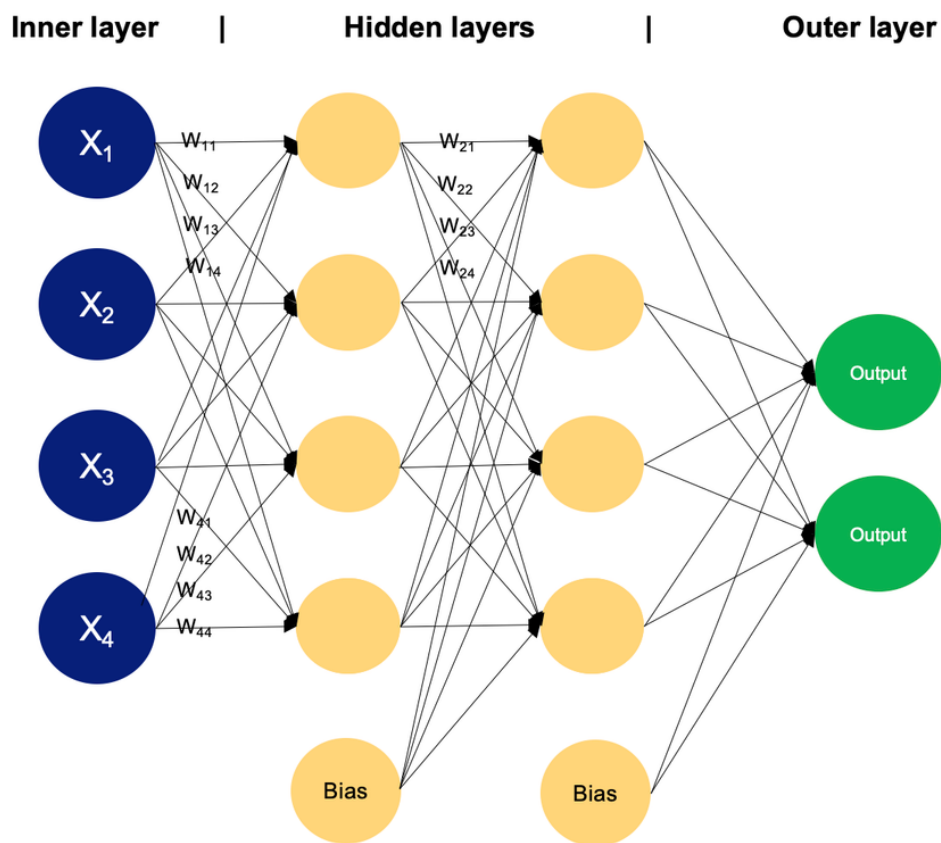


Figure 4 : exemple de MLP à 2 couches cachées

La propagation avant (*Forward Propagation*) est le processus par lequel les données d'entrée traversent le réseau pour produire une prédiction. Pour une couche donnée l comprenant m neurones et recevant un vecteur d'activation de la couche précédente $\mathbf{a}^{[l-1]} \in \mathbb{R}^n$, l'activation de chaque neurone j (où $j = 1, \dots, m$) s'effectue en deux étapes mathématiques distinctes :

1. La combinaison affine : Le neurone calcule la somme pondérée de ses entrées, à laquelle il additionne un biais propre. Ce potentiel d'activation net $z_j^{[l]}$ s'écrit :

$$z_j^{[l]} = \sum_{i=1}^n w_{ji}^{[l]} a_i^{[l-1]} + b_j^{[l]}$$

Où $w_{ji}^{[l]}$ désigne le poids synaptique reliant le neurone i de la couche $l - 1$ au neurone j de la couche l , et $b_j^{[l]}$ représente le biais du neurone j de la couche l .

2. La transformation non-linéaire : Ce potentiel net est ensuite projeté par une fonction d'activation $g^{[l]}$ pour donner l'activation finale $a_j^{[l]}$:

$$a_j^{[l]} = g^{[l]}(z_j^{[l]})$$

À l'échelle de la couche l dans son intégralité, ces opérations individuelles sont vectorisées sous forme matricielle afin d'exploiter au maximum le calcul parallèle :

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = g^{[l]}(\mathbf{z}^{[l]})$$

Où :

- $\mathbf{W}^{[l]} \in \mathbb{R}^{m \times n}$ est la matrice des poids de la couche l .
- $\mathbf{b}^{[l]} \in \mathbb{R}^m$ est le vecteur des biais de la couche l .
- $\mathbf{z}^{[l]} \in \mathbb{R}^m$ est le vecteur des potentiels d'activation nets.
- $\mathbf{a}^{[l]} \in \mathbb{R}^m$ est le vecteur d'activation de la couche l , avec la convention que pour la couche d'entrée ($l = 0$), $\mathbf{a}^{[0]} = \mathbf{x}$, le vecteur des caractéristiques brutes.

3.4.2. Mécanisme d'Apprentissage et Rétropropagation

L'apprentissage d'un MLP consiste à déterminer les matrices de poids $\mathbf{W}$ et les vecteurs de biais $\mathbf{b}$ qui minimisent une fonction de coût globale $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$, mesurant l'erreur entre la prédiction du réseau $\hat{\mathbf{y}}$ et la cible réelle $\mathbf{y}$.

Ce problème d'optimisation non-linéaire est résolu itérativement par l'algorithme de la **rétropropagation du gradient** (*Backpropagation*), qui s'appuie sur le théorème de dérivation des fonctions composées (*Chain Rule*). Pour mettre à jour un poids $w_{ji}^{[l]}$, il faut calculer la dérivée partielle de la fonction de coût par rapport à ce poids :

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{[l]}} = \frac{\partial \mathcal{L}}{\partial z_j^{[l]}} \cdot \frac{\partial z_j^{[l]}}{\partial w_{ji}^{[l]}}$$

En définissant l'erreur locale du neurone j à la couche l par la variable intermédiaire

$\delta_j^{[l]} = \frac{\partial \mathcal{L}}{\partial z_j^{[l]}}$, et sachant d'après la combinaison affine que $\frac{\partial z_j^{[l]}}{\partial w_{ji}^{[l]}} = a_i^{[l-1]}$, l'expression se simplifie sous la forme :

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{[l]}} = \delta_j^{[l]} \cdot a_i^{[l-1]}$$

Le calcul de l'erreur locale $\delta_j^{[l]}$ s'effectue de manière récursive, en partant de la couche finale de sortie L et en remontant vers les couches initiales. Pour une couche cachée intermédiaire l , cette erreur se propage depuis la couche supérieure $l + 1$ selon la relation :

$$\delta_j^{[l]} = \left(\sum_k \delta_k^{[l+1]} w_{kj}^{[l+1]} \right) \cdot g'^{[l]}(z_j^{[l]})$$

Une fois l'ensemble des gradients partiels évalué pour l'ensemble du réseau, les paramètres sont mis à jour via un algorithme d'optimisation (comme l'optimiseur Adam sélectionné pour ce projet) selon les règles de descente de gradient :

$$\begin{aligned} \mathbf{W}^{[l]} &\leftarrow \mathbf{W}^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} \\ \mathbf{b}^{[l]} &\leftarrow \mathbf{b}^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} \end{aligned}$$

Où α désigne le taux d'apprentissage (*Learning Rate*).

3.4.3. Les Limites Structurelles du MLP face aux Données Visuelles

Bien que le MLP soit un approximateur universel théorique, son application pratique à la classification d'images révèle deux failles majeures :

1. **La rupture de la topologie spatiale** : Une image est par nature une matrice bidimensionnelle ou tridimensionnelle contenant des corrélations locales fortes entre pixels adjacents. Pour alimenter un MLP, cette structure doit être aplatie (*Flatten*) sous forme d'un vecteur unidimensionnel. Cette opération détruit la géométrie de l'image : un pixel perd ses relations de voisinage vertical et diagonal, forçant le réseau à réapprendre laborieusement des concepts géométriques élémentaires.
2. **L'explosion de la dimensionnalité des paramètres** : Considérons les images de ce projet, standardisées à une résolution de 256×256 pixels avec 3 canaux de couleur (RGB). Leur aplatissement engendre un vecteur d'entrée $\mathbf{x} \in \mathbb{R}^{196\,608}$. Si la première couche cachée du MLP comporte seulement 1024 neurones, la matrice de poids associée $\mathbf{W}^{[1]}$ requiert l'optimisation de :

$$1024 \times 196\,608 = 2\,013\,265\,920 \text{ paramètres.}$$

Dépasser les deux milliards de paramètres dès la première couche logicielle crée une architecture impossible à généraliser sans un sur-apprentissage (*overfitting*) immédiat et massif, exigeant simultanément une puissance de calcul et une mémoire vidéo hors de portée.

3.5. Les Réseaux de Neurones Convolutifs (CNN)

Pour surmonter les limitations intrinsèques du MLP, les Réseaux de Neurones Convolutifs (CNN) introduisent un biais inductif fort basé sur les propriétés physiques des images. Au lieu de connecter chaque unité à l'ensemble de l'espace d'entrée, le CNN impose deux contraintes mathématiques rigoureuses : la **connectivité locale** et le **partage des poids**.

3.5.1. Le Principe de Connectivité Locale

Au lieu d'analyser l'image dans sa globalité d'un seul coup, un neurone convolutif n'observe qu'une petite sous-région spatiale de l'entrée, appelée son **champ récepteur** (*Receptive Field*).

Si l'on vulgarise visuellement, la couche de convolution agit comme une lampe de poche qui éclaire une zone de 3×3 pixels sur l'image. Le neurone analyse exclusivement les relations entre ces 9 pixels pour y détecter un motif local (un bout de ligne, un contraste brutal, un gradient coloré). Cette approche reflète parfaitement la construction de notre propre cortex visuel : la compréhension d'une image complexe naît de l'assemblage d'une multitude d'observations locales.

3.5.2. L'Opération de Convolution

Au lieu d'affecter des poids uniques à chaque région de l'image, un CNN utilise un jeu de filtres qui balaient l'intégralité de la surface de l'entrée. Tous les neurones d'une même carte de caractéristiques (*Feature Map*) partagent ainsi les mêmes poids synaptiques.

Mathématiquement, l'opération réalisée dans une couche convolutive bidimensionnelle standard est une intercorrélacion croisée discrète. Soit une matrice d'entrée à un seul canal $\mathbf{I} \in \mathbb{R}^{H \times W}$ et un noyau de convolution $\mathbf{K} \in \mathbb{R}^{h \times w}$. La valeur de la carte de caractéristiques résultante $\mathbf{S}$ à la coordonnée spatiale (i, j) s'exprime par la double somme :

$$S(i, j) = (\mathbf{I} * \mathbf{K})(i, j) = \sum_{m=0}^{h-1} \sum_{n=0}^{w-1} I(i + m, j + n) \cdot K(m, n) + b$$

Où $b \in \mathbb{R}$ est le biais partagé pour ce filtre.

Dans le cas réel d'une image couleur ou d'une couche intermédiaire possédant C canaux d'entrée, le volume d'entrée devient $\mathbf{I} \in \mathbb{R}^{C \times H \times W}$. Le noyau possède alors également C canaux, et sa dimension est $\mathbf{K} \in \mathbb{R}^{C \times h \times w}$. L'opération intègre une sommation supplémentaire sur l'ensemble des canaux :

$$S(i, j) = \sum_{c=0}^{C-1} \sum_{m=0}^{h-1} \sum_{n=0}^{w-1} I(c, i + m, j + n) \cdot K(c, m, n) + b$$

Si la couche convolutive applique N filtres distincts pour extraire des caractéristiques différentes, elle génère un volume de sortie comportant N cartes de caractéristiques.

Gain d'optimisation : Pour un filtre de taille 3×3 appliqué à l'image d'entrée de $256 \times 256 \times 3$, le nombre de paramètres à optimiser pour une carte de caractéristiques n'est que de $3 \times 3 \times 3 + 1 = 28$ paramètres, quelle que soit la taille de la carte de sortie. Le partage des poids réduit de manière drastique l'empreinte mémoire et le risque d'overfitting par rapport au MLP.

3.5.3. Les Cartes de Caractéristiques (*Feature Maps*)

Une couche de convolution n'applique pas un seul filtre, mais des dizaines (16, puis 32, 64, etc., dans notre architecture *Baseline*).

- Les premières couches vont générer des cartes de caractéristiques de bas niveau : un filtre va s'activer uniquement sur les contours verticaux, un autre sur les contours horizontaux, un autre sur les textures tachetées.
- Les couches intermédiaires, en observant les cartes produites par les couches précédentes, vont combiner ces lignes pour former des géométries : des cercles (les yeux), des triangles (les oreilles).
- Les couches profondes assembleront ces formes pour détecter des concepts sémantiques complexes : un museau de chien ou la silhouette d'un chat.

L'image initiale (3 canaux RGB) est ainsi transformée en un cube de données beaucoup plus "profond" (par exemple 256 canaux), où chaque canal représente la présence ou l'absence d'un motif précis dans l'image.

3.5.4. La Couche de Pooling : Synthèse et Robustesse Spatiale

Après avoir extrait ces caractéristiques par convolution et appliqué une activation non-linéaire (ReLU) pour éliminer les valeurs négatives, le réseau utilise des couches de sous-échantillonnage (*Pooling*).

Le **Max Pooling**, massivement utilisé dans ce projet, consiste à découper la carte de caractéristiques en blocs de 2×2 et à ne conserver que la valeur maximale de chaque bloc :

$$P(i, j) = \max_{(m, n) \in \mathcal{N}(i, j)} S(m, n)$$

Visuellement et conceptuellement, cette opération répond à deux nécessités :

1. **Réduction dimensionnelle** : Elle divise par 4 le nombre de pixels à traiter. L'image de 256×256 devient 128×128 , puis 64×64 , etc. On compresse l'espace physique pour laisser place à la profondeur sémantique.
2. **Invariance spatiale locale** : En ne gardant que l'activation maximale d'une zone de 2×2 , on indique au réseau : *"Peu m'importe que le contour de l'œil soit exactement au pixel (45, 45) ou au pixel (46, 46). S'il est dans cette zone, retiens simplement qu'il est présent."* Cette perte volontaire de la précision géographique rend le réseau incroyablement robuste aux déformations, aux légères rotations et aux variations morphologiques entre un chat et un chien.

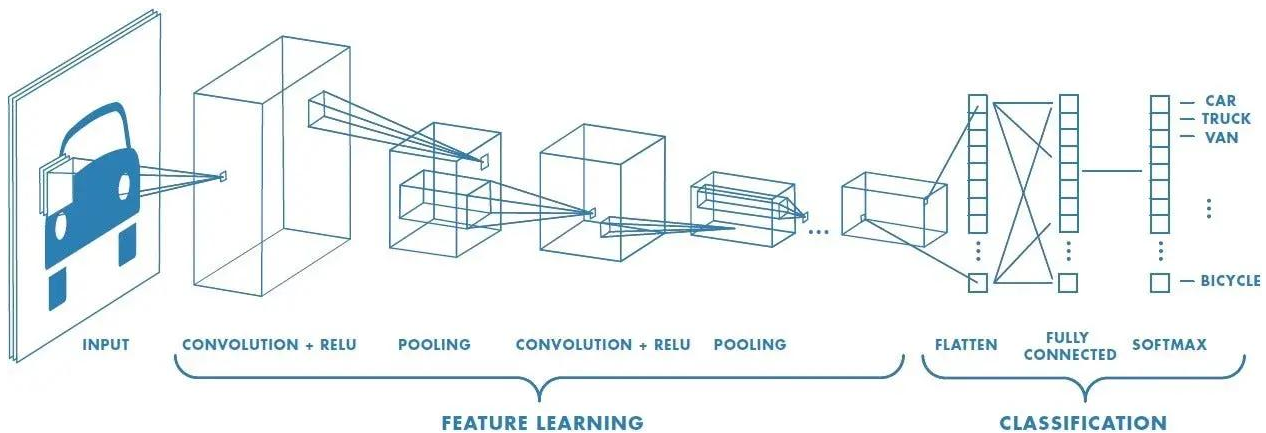


Figure 5 : représentation d'un CNN de façon schématique

4. Développement et Implémentation

4.1. Configuration de l'Environnement de Travail

La première étape indispensable de la phase de développement a consisté à mettre en place un environnement de travail robuste, isolé et reproductible, spécialement adapté au Deep Learning. Pour cela, nous avons choisi d'utiliser **Conda** comme gestionnaire d'environnements virtuels et de paquets.

4.1.1. Création de l'environnement Conda

Afin d'éviter tout conflit de versions avec d'autres projets sur la machine de développement, un environnement virtuel isolé a été créé spécifiquement pour l'UV AC20, avec une version stable de Python (3.10) :

```
conda create -n ac20 python=3.10
conda activate ac20
```

4.1.2. Installation de PyTorch avec support CUDA

L'entraînement de Réseaux de Neurones Convolutifs (CNN) sur des milliers d'images est une tâche extrêmement gourmande en ressources de calcul. Utiliser le processeur (CPU) aurait rendu les temps d'entraînement prohibitifs.

Nous avons donc configuré le framework **PyTorch** pour qu'il tire parti de l'accélération matérielle fournie par la carte graphique (GPU) de la machine, une NVIDIA RTX 3050. L'installation a été réalisée avec la prise en charge de CUDA pour exploiter les cœurs de calcul du GPU :

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
```

L'ensemble des autres bibliothèques nécessaires à la Data Science et à la visualisation (Jupyter, Pandas, NumPy, Matplotlib, Scikit-learn, etc.) a également été installé au sein de cet environnement. Pour garantir la reproductibilité totale du projet par d'autres acteurs (notamment pour l'évaluation), la configuration complète a été exportée dans un fichier `environment.yml`.

4.1.3. Vérification de l'accélération matérielle (GPU)

Pour s'assurer que PyTorch communique correctement avec les pilotes de la carte graphique et peut utiliser l'API CUDA, un test de validation a été effectué dans l'interpréteur Python. La fonction booléenne `torch.cuda.is_available()` a été exécutée.

```
print(f'Using device: {device}')
```

```
Using device: cuda
```

Figure 6 : Vérification de la disponibilité de CUDA sous PyTorch.

Comme l'illustre la capture d'écran ci-dessus, le système renvoie bien True. Cela confirme que notre environnement est parfaitement configuré et prêt à utiliser la puissance de calcul de la RTX 3050 pour accélérer nos futurs entraînements de modèles.

4.2. Préparation des Données et Analyse Exploratoire (EDA)

Lors de cette première étape de traitement, nous avons analysé le dataset Kaggle « Dogs vs Cats » dans le but de nettoyer les données brutes. À l'aide du script `_cleaner.ipynb`, nous avons parcouru l'ensemble des images pour identifier celles qui étaient corrompues ou illisibles par la bibliothèque.

Résultats de l'analyse et du nettoyage : - Images corrompues détectées et supprimées : 4. - Hauteur moyenne : ~361 pixels. - Largeur moyenne : ~404 pixels.

```
analyse_data("../data") #j'appelle la fonction analyse_data en lui passant le chemin du dossier data
✓ 9.4s

Dossier : Cat
Erreur lors de l'ouverture de l'image 666.jpg : cannot identify image file '../data\\Cat\\666.jpg'
Erreur lors de l'ouverture de l'image Thumbs.db : cannot identify image file '../data\\Cat\\Thumbs.db'
Dossier : Dog
Erreur lors de l'ouverture de l'image 11702.jpg : cannot identify image file '../data\\Dog\\11702.jpg'
c:\Users\abdall\anaconda3\envs\ac20\lib\site-packages\PIL\TiffImagePlugin.py:949: UserWarning: Truncated File Read
warnings.warn(str(msg))
Erreur lors de l'ouverture de l'image Thumbs.db : cannot identify image file '../data\\Dog\\Thumbs.db'
Nombre d'images corrompues : 4
Hauteur moyenne des images : 361.04701128270784
Largeur moyenne des images : 404.51192286148677
```

Figure 7 : Affichage des résultats du nettoyage dans le Notebook.

Ces statistiques descriptives sont cruciales. Elles nous permettront de choisir judicieusement la taille de redimensionnement (par exemple 224x224 ou 256x256) lors de la création de nos *DataLoaders* pour que l'entrée de notre réseau de neurones convolutif soit standardisée.

4.2.1. Séparation des Données (Split Train/Val/Test)

Pour entraîner correctement notre modèle et évaluer ses performances de manière impartiale, nous avons divisé notre jeu de données "propre" en trois sous-ensembles distincts. Cette étape garantit que le modèle est évalué sur des images qu'il n'a jamais vues pendant l'apprentissage, évitant ainsi de masquer un éventuel sur-apprentissage (overfitting).

Nous avons développé un script Python dédié (`scripts/prepare_data.py`) qui effectue cette séparation de manière aléatoire mais reproductible (en fixant une "seed" ou graine aléatoire). La répartition choisie est la suivante :

- **Ensemble d'entraînement (Train)** : 80% des données. Utilisé pour ajuster les poids du modèle.
- **Ensemble de validation (Val)** : 10% des données. Utilisé pour régler les hyperparamètres et surveiller l'overfitting à chaque "epoch".
- **Ensemble de test (Test)** : 10% des données. Conservé précieusement pour l'évaluation finale et globale du modèle.

Le script copie physiquement les images dans une nouvelle arborescence structurée (data/clean/train/, data/clean/val/, data/clean/test/), ce qui simplifie grandement leur chargement ultérieur par le framework PyTorch.

4.2.2. Pipeline de Chargement (DataLoaders et Transformations)

Une fois les dossiers structurés, nous avons utilisé l'API `torch.utils.data` et `torchvision.datasets` dans notre notebook d'expérimentation (02_baseline_cnn.ipynb) pour créer un pont entre les images sur le disque dur et la carte graphique.

L'utilisation de la classe `ImageFolder` de `torchvision` s'est avérée particulièrement pertinente, car elle infère automatiquement les étiquettes (labels "Cat" et "Dog") à partir du nom des sous-dossiers.

Pour préparer les images à entrer dans le réseau de neurones, plusieurs transformations ont été appliquées lors du chargement :

- **Redimensionnement** : Toutes les images ont été redimensionnées à **256x256 pixels**. Ce choix a été fait en tenant compte de l'analyse exploratoire précédente (taille moyenne originelle autour de 360x400), afin de conserver un maximum de détails spatiaux tout en utilisant une dimension qui est une puissance de 2, idéale pour les opérations de "Pooling" successives.
- **Conversion Tensor (ToTensor)** : Transformation des images PIL (pixels entre 0 et 255) en tenseurs PyTorch (valeurs flottantes entre 0.0 et 1.0).
- **Normalisation (Normalize)** : Centrage des valeurs des pixels autour de 0 (entre -1.0 et 1.0) en utilisant une moyenne de `[0.5, 0.5, 0.5]` et un écart-type de `[0.5, 0.5, 0.5]` pour chaque canal de couleur (RGB). Cette étape mathématique est cruciale pour assurer la stabilité numérique et la vitesse de convergence de l'algorithme d'optimisation lors de l'apprentissage.

Les `DataLoaders` ont été configurés avec une taille de lot (Batch Size) de 32 images. L'ensemble d'entraînement est mélangé aléatoirement (`shuffle=True`) à chaque époque pour éviter que le modèle n'apprenne l'ordre des images, tandis que les ensembles de validation et de test ne le sont pas, car cela n'affecte pas l'évaluation.

4.3. Expérience 1 : Le Modèle Baseline (“From Scratch”)

4.3.1. Architecture du Réseau de Neurones Convolutif (CNN)

Pour établir un point de référence (Baseline), nous avons implémenté une architecture CNN artisanale de toutes pièces (“From Scratch”) en héritant de la classe `nn.Module` de PyTorch. L’objectif de ce modèle n’est pas d’atteindre l’état de l’art immédiatement, mais de valider la viabilité de notre pipeline complet d’apprentissage.

L’architecture conçue suit un motif classique d’extraction de caractéristiques (“Feature Extraction”) suivi d’une classification (“Classifier”). L’extracteur est composé de **5 blocs convolutifs successifs**.

Composition de chaque bloc convolutif :

- Une couche de **Convolution 2D** (`nn.Conv2d`) avec un filtre (kernel) de taille 3x3 et un padding de 1 pour conserver les dimensions de l’image.
- Une fonction d’activation non-linéaire **ReLU** (`nn.ReLU`).
- Une couche de regroupement maximal **Max Pooling 2D** (`nn.MaxPool2d`) avec une fenêtre de 2x2, qui divise la dimension spatiale par deux.

En appliquant le principe architectural empirique voulant que l’on augmente la “profondeur” (nombre de filtres) à mesure que l’on réduit la dimension “spatiale” (hauteur et largeur), le nombre de filtres double à chaque bloc : 16, puis 32, 64, 128, pour finir à **256 filtres** dans le cinquième et dernier bloc.

Calcul des dimensions et Classificateur : L’image initiale en entrée ayant une dimension de 256x256 pixels, elle subit 5 divisions successives par 2 au travers des couches de Max Pooling (256 → 128 → 64 → 32 → 16 → 8). À la sortie de l’extracteur, nous obtenons donc un cube de données (“Feature Map”) de dimension **256 canaux × 8 hauteurs × 8 largeur**.

La partie classification (`self.classifier`) débute par un aplatissement (`nn.Flatten()`) de ce cube pour obtenir un vecteur 1D de 16 384 éléments (256 × 8 × 8). Ce vecteur traverse ensuite deux couches de neurones denses (Fully Connected) : - Une première couche dense réduisant la dimensionnalité à 256 neurones, activée par une fonction ReLU. - Une couche dense finale (`nn.Linear(256, 2)`) donnant les 2 sorties “brutes” (logits) correspondant aux probabilités d’appartenir à la classe “Chien” ou à la classe “Chat”.

Cette conception, programmée de manière modulaire à l’aide d’une boucle Python, permet une instanciation propre et claire du réseau. Le modèle a ensuite été transféré avec succès vers la mémoire de la carte graphique (GPU) pour préparer l’étape d’entraînement.

4.3.2. Entraînement et Analyse des Résultats (Baseline)

Pour l’entraînement de ce modèle de base, nous avons sélectionné l’optimiseur **Adam** avec un taux d’apprentissage (learning rate) de 0.001, couplé à la fonction de perte `CrossEntropyLoss` adaptée aux problèmes de classification multiclassés.

Un mécanisme d'**Early Stopping** a été mis en place avec une “patience” de 5 époques. Son but est d'interrompre l'apprentissage dès que la perte de validation (`val_loss`) cesse de s'améliorer, évitant ainsi de poursuivre des calculs inutiles et de dégrader la capacité de généralisation du modèle.

Résultats de l'entraînement : Le modèle s'est entraîné rapidement, mais l'arrêt prématuré s'est déclenché à l'époque 12. Le meilleur modèle (sauvegardé à l'époque 7) a atteint les performances suivantes :

- **Loss de Validation :** 0.3309
- **Accuracy de Validation :** 88.67%
-

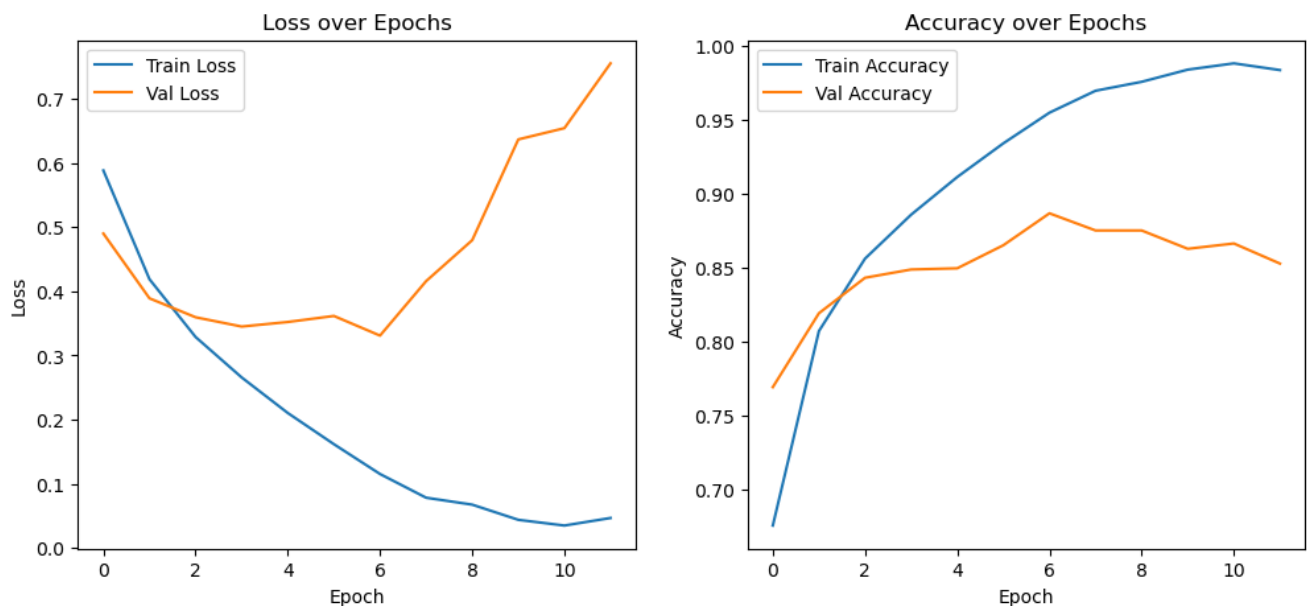


Figure 8 : Courbes d'apprentissage du modèle Baseline

Bien que la précision sur l'ensemble d'entraînement continue de grimper pour approcher les 98.37% à l'époque 12, la perte de validation, après avoir atteint son minimum à l'époque 4, recommence à augmenter. C'est le symptôme classique du **sur-apprentissage**. Le modèle commence à “apprendre par cœur” les images d'entraînement et perd sa capacité à généraliser sur des données inconnues.

Ces résultats valident notre architecture de base mais soulignent la nécessité d'introduire des mécanismes de régularisation pour améliorer les performances globales et la robustesse du modèle.

4.4. Expérience 2 : Optimisation et Régularisation

À la suite des observations de l'expérience précédente, une phase d'optimisation a été menée, l'objectif était d'atténuer le phénomène d'overfitting et d'améliorer la convergence en appliquant plusieurs stratégies :

1. **Data Augmentation (Augmentation de données)** : Application de transformations aléatoires (rotations, retournements horizontaux) sur les images d'entraînement. Cela force le réseau à apprendre des caractéristiques invariantes.
2. **Couches de Dropout** : Nous avons intégré un Dropout de 0.5 dans le classificateur pour empêcher la co-adaptation des neurones.
3. **Batch Normalization** : Des couches de normalisation de lot ont été ajoutées après chaque convolution pour stabiliser l'apprentissage et permettre une convergence plus rapide.

Résultats du modèle optimisé : L'ajout de ces techniques a radicalement transformé les performances :

- **Loss de Validation : 0.1108**
- **Accuracy de Validation : 95.76%**

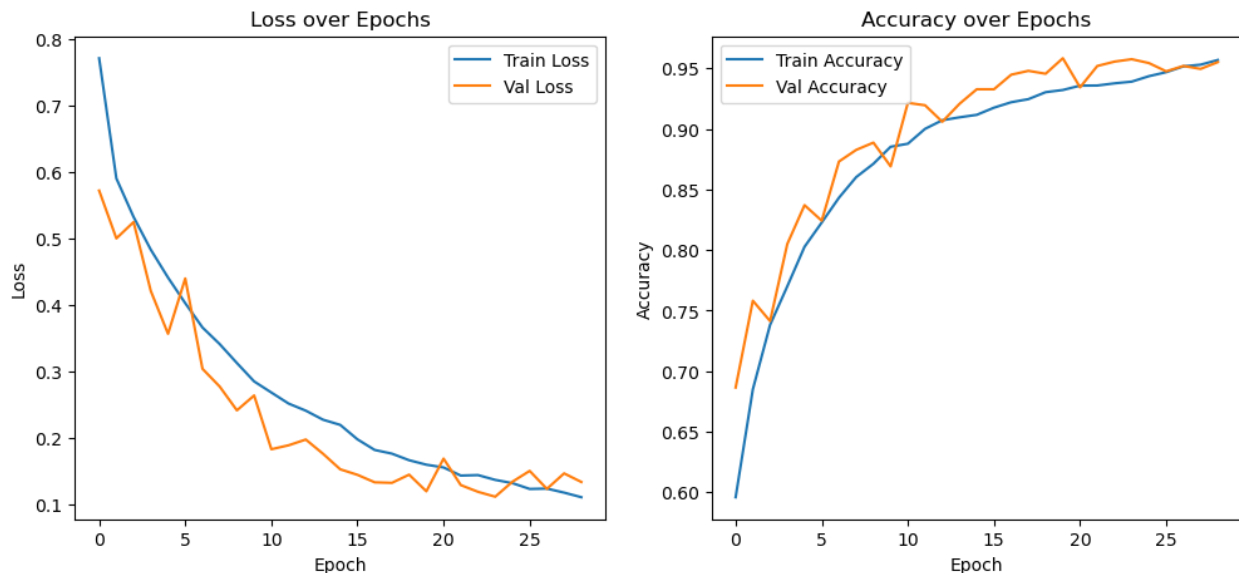


Figure 9 : Courbes d'apprentissage du modèle Optimisé

De plus, la Batch Normalization a permis de réduire légèrement le temps de traitement par image, passant de 0.0078s à **0.0072s**.

Nous allons donc comparée nos deux modèles sur le même jeu de donnée de test qui n'as jamais été vue par les deux.

4.5. Analyse Comparative : Optimisé VS Baseline

Pour garantir la validité scientifique de cette analyse comparative, il est impératif de s'affranchir de tout biais d'évaluation. Au cours des phases d'apprentissage respectives, l'ensemble de validation a été activement sollicité, d'une part pour paramétrer l'architecture (ajustement du taux de *Dropout*), et d'autre part pour interrompre l'apprentissage au moment optimal via le mécanisme d'*Early Stopping*. Évaluer les performances finales sur ce même sous-ensemble introduirait inévitablement un biais d'optimisation (fuite d'information indirecte).

C'est pourquoi la confrontation des modèles *Baseline* et Optimisé s'effectue exclusivement sur le jeu de test. Représentant 10 % du corpus initial, ce sous-ensemble a été strictement sanctuarisé tout au long du cycle de développement. Il n'a jamais influencé la mise à jour des poids synaptiques ni le choix des hyperparamètres, faisant ainsi office de données totalement inédites pour simuler le comportement des modèles en conditions réelles.

L'évaluation sur ce jeu de test neutre s'articule autour de deux axes métriques :

1. L'efficacité computationnelle : Évaluée par le temps d'inférence moyen par image.
 2. La performance prédictive : Bien que l'Exactitude globale (*Accuracy*) donne une première indication, une analyse rigoureuse exige une granularité plus fine. À partir de la matrice de confusion, nous avons extrait trois métriques statistiques fondamentales, basées sur les Vrais Positifs (TP), Faux Positifs (FP) et Faux Négatifs (FN) pour chaque classe :
- La Précision (*Precision*) : Elle évalue la proportion de prédictions correctes parmi l'ensemble des prédictions positives effectuées par le modèle. Cette métrique indique à quel point nous pouvons faire confiance au modèle lorsqu'il affirme qu'une image appartient à une classe donnée et quantifie sa capacité à limiter les fausses alarmes (Faux Positifs).

$$Precision = \frac{TP}{TP + FP}$$

- Le Rappel (*Recall* ou Sensibilité) : Il mesure la proportion d'exemples positifs réels qui ont été correctement identifiés par l'algorithme. Le Rappel est crucial pour s'assurer que le réseau minimise les omissions (Faux Négatifs).

$$Recall = \frac{TP}{TP + FN}$$

- Le F1-Score : Il s'agit de la moyenne harmonique entre la Précision et le Rappel. Le F1-score constitue la métrique de référence pour attester de la robustesse globale et de l'équilibre du classificateur.

$$F1-score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Résultats et Analyse Causale :

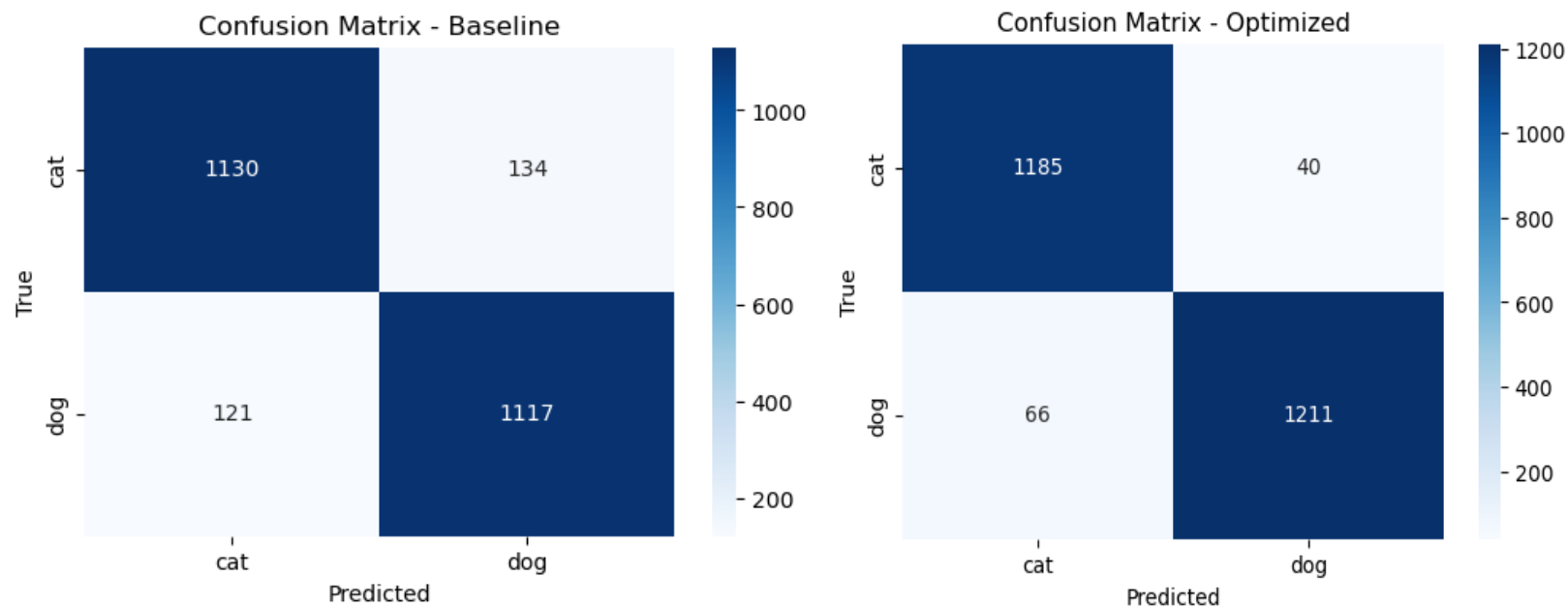


Figure 10 : Matrice de confusion du modèle Baseline et Optimisée sur le jeu de test

Tableau comparatif des métriques (Extrait des matrices de confusion) :

Modèle	Accuracy	Temps par Image (s)
Baseline	89.81 %	0.0078
Optimisé	95.76 %	0.0072

Modèle	Classe	Précision	Rappel	F1-Score
Baseline	Chat	90 %	89 %	90 %
	Chien	89 %	90 %	90 %
Optimisé	Chat	95 %	97 %	96 %
	Chien	97 %	95 %	96 %

Analyse de l'efficacité computationnelle : Concernant la vitesse d'exécution, les durées d'inférence mesurées (0.0078 s pour la *Baseline* contre 0.0072 s pour le modèle optimisé) restent dans le même ordre de grandeur. Les variations relevées (inférieures à la milliseconde) ne sont pas statistiquement significatives et relèvent de la charge matérielle lors du test. Nous pouvons toutefois affirmer que l'ajout des couches de *Batch Normalization* et de *Dropout* n'a engendré aucune surcharge computationnelle pénalisante en phase d'inférence.

Analyse des performances prédictives : L'amélioration significative de l'Exactitude globale, passant de 89.81 % à 95.76 %, valide de manière empirique l'impact de nos choix architecturaux. L'action combinée de la *Data Augmentation* et du *Dropout* a efficacement brisé les co-adaptations des neurones, forçant le réseau à apprendre des caractéristiques plus robustes plutôt que de mémoriser le jeu d'entraînement.

L'exploitation des métriques fines illustre parfaitement ce gain de généralisation :

- La réduction massive des omissions (Évolution du Rappel) : Le bond le plus spectaculaire concerne le Rappel de la classe "Chat", qui grimpe de 89 % (*Baseline*) à 97 % (Optimisé). Concrètement, le modèle de base générait un nombre important de faux négatifs sur les félins. L'introduction de la *Data Augmentation* a forcé le réseau à s'exposer à des orientations, échelles et variations morphologiques diverses, ce qui lui a permis de détecter quasi systématiquement la présence d'un chat dans l'image.
- L'équilibre global (Évolution du F1-Score) : Le F1-score bondit de 90 % à 96 % pour les deux classes. Cette évolution harmonieuse démontre que l'optimisation n'a pas sacrifié la Précision au profit du Rappel : le modèle est devenu non seulement plus sensible (il rate moins de sujets), mais également beaucoup plus fiable lorsqu'il émet une prédiction.

4.6. Expérience 3 : Approche par Transfer Learning (ResNet50)

4.6.1. Justification de l'approche et préparation des données

Afin d'éprouver les limites de notre architecture artisanale, nous avons confronté notre modèle à l'état de l'art en utilisant la technique du *Transfer Learning* (Apprentissage par Transfert). Cette méthode consiste à exploiter les paramètres (poids et biais) d'un modèle profond pré-entraîné sur un corpus massif — en l'occurrence ImageNet, contenant plus d'un million d'images et un millier de classes — pour résoudre notre problème de classification binaire.

Le modèle sélectionné est le ResNet50 (Residual Network à 50 couches), célèbre pour l'introduction des connexions résiduelles (Skip Connections) qui permettent de pallier le problème de disparition du gradient dans les réseaux très profonds.

L'utilisation de cette architecture standardisée a nécessité une adaptation de notre pipeline de données :

- Redimensionnement strict : L'architecture ResNet s'attend à recevoir des tenseurs de dimensions géométriques précises. Toutes les images ont été redimensionnées à 224x224 pixels.
- Normalisation ImageNet : Contrairement au centrage classique utilisé pour notre *Baseline*, les données ont été normalisées en utilisant les moyennes (0.485, 0.456, 0.406) et écarts-types (0.229, 0.224, 0.225) spécifiques au jeu de données ImageNet pour les canaux RGB. Cette étape garantit que les activations des premières couches du réseau opèrent dans la plage de valeurs pour laquelle elles ont été initialement optimisées.

4.6.2. Architecture et Stratégie d'entraînement en deux phases

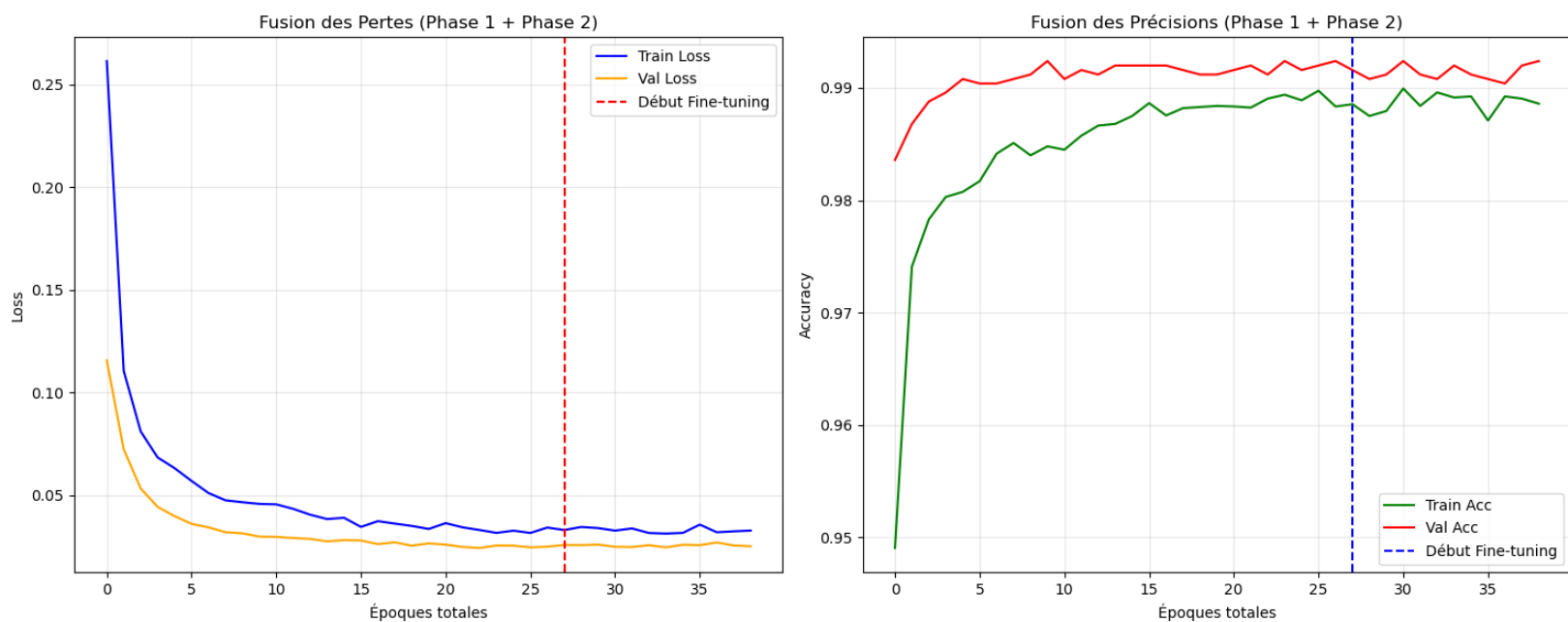
Pour adapter le ResNet50 à notre problème, l'extracteur de caractéristiques a été conservé intact, tandis que la couche finale de classification (Fully Connected), initialement prévue pour 1000 classes, a été remplacée par une nouvelle couche dense comportant uniquement 2 neurones en sortie (`nn.Linear(num_features, 2)`).

Afin d'éviter la destruction des poids pré-entraînés par des gradients trop importants lors des premières itérations (phénomène d'oubli catastrophique), une stratégie d'entraînement itérative en deux phases a été déployée :

- Phase 1 (Feature Extraction) : Congélation (*Freezing*) de l'ensemble des couches convolutives. Seuls les paramètres de la nouvelle couche de classification ont été soumis à l'optimiseur Adam. Avec un taux d'apprentissage de 0.0001, le modèle a rapidement convergé, déclenchant l'arrêt prématuré (Early Stopping) après 28 époques avec une précision de validation exceptionnelle de 99.24 % et une perte de 0.0248.
- Phase 2 (Fine-Tuning) : Décongélation de l'ensemble du réseau pour un ajustement fin des paramètres. Le taux d'apprentissage a été drastiquement réduit à 0.00001. Cette phase s'est avérée extrêmement brève, l'Early Stopping s'activant dès la 11ème époque, stabilisant la précision de validation à 99.24 % pour une perte de 0.0250.

Cette dynamique d'entraînement prouve que les primitives visuelles extraites par les couches inférieures de ResNet50 étaient déjà parfaitement discriminantes pour notre tâche, rendant le *Fine-Tuning* des couches profondes quasi superflu d'un point de vue statistique.

Figure 11 : Courbes d'apprentissage du modèle en Transfer Learning



4.6.3. Résultats finaux et Évaluation critique

L'évaluation finale du modèle ResNet50 sur le jeu de test sanctuarisé a livré les résultats suivants :

Modèle	Accuracy	Temps par Image (s)	Précision (Chat/Chien)	Rappel (Chat/Chien)	F1-Score
Transfer Learning (ResNet50)	99.16 %	0.0106	1.00 / 0.99	0.99 / 1.00	99 %

Analyse comparative avec la Baseline optimisée :

L'approche par Transfer Learning frôle la classification parfaite. La matrice de confusion révèle que sur un échantillon de 2502 images (1260 chats et 1242 chiens), la proportion d'erreurs est devenue négligeable.

Néanmoins, une analyse d'ingénierie rigoureuse impose de contextualiser ce score. Si le ResNet50 surclasse notre modèle CNN optimisé en termes de performance pure (+3.4 %

d'Accuracy), il introduit un coût computationnel non négligeable. Le temps d'inférence moyen par image passe de 0.0072 s (pour le CNN optimisé) à 0.0106 s (pour le ResNet50), ce qui représente une augmentation du temps de calcul de près de 47 %.

Le choix entre ces deux algorithmes dans un environnement de production dépendra donc ultimement du cahier des charges : le modèle artisanal pour une application en temps réel à ressources contraintes (Edge Computing), et le modèle par Transfer Learning pour une analyse où la tolérance à l'erreur est proche de zéro.

4.7. Expérience 4 : Ouverture vers la Détection d'Objets (YOLOv8)

4.7.1. Motivation et Rupture de Paradigme

Pour clore notre campagne expérimentale, nous avons souhaité ouvrir notre problématique aux enjeux de la vision par ordinateur en temps réel en introduisant l'architecture YOLOv8 (*You Only Look Once*, version *Nano* : yolov8n.pt).

L'objectif de cette expérience n'est pas seulement de comparer des scores d'exactitude, mais de mesurer l'apport d'un changement fondamental de paradigme algorithmique. Alors que nos architectures précédentes effectuent une classification globale (affectation d'une étiquette unique à l'image entière), YOLOv8 réalise une tâche duale : la prédiction de coordonnées de boîtes englobantes (*Bounding Boxes Regression*) par rapport à des ancres dynamiques et la classification des zones localisées.

4.7.2. Implémentation du Protocole de Test

N'ayant pas réentraîné YOLOv8 sur notre corpus, nous l'avons évalué en mode *Zero-Shot Inference* (utilisation directe des poids optimisés sur le jeu de données généraliste COCO). Les classes d'intérêt ont été mappées syntaxiquement : l'indice local 0 (Cat) correspond à l'indice 15 de COCO, et l'indice local 1 (Dog) à l'indice 16.

Le pipeline de données a introduit des contraintes géométriques et logiques strictes :

- Interpolation spatiale : Les images du jeu de test ont été redimensionnées en 640×640 pixels afin de respecter la résolution native de l'extracteur de caractéristiques de YOLOv8.
- Gestion des non-détections : Contrairement aux réseaux de neurones convolutifs classiques qui distribuent de manière forcée une probabilité sur les sorties (Softmax/Logits), YOLO peut conclure à l'absence de cible si aucune boîte ne franchit

le seuil de confiance statistique. Ces cas ont été explicitement capturés et enregistrés sous une classe sentinelle -1 (*None*) afin de ne pas masquer les faiblesses du modèle.

4.7.3. Résultats et Confrontation Critique

L'évaluation sur le jeu de test sanctuarisé a donné une précision globale (incluant les non-détections) de 78,18 %.

Modèle	Approche	Résolution	Test Accuracy	Avantage Industriel
Baseline	CNN <i>From Scratch</i>	256 × 256	89,81 %	Léger, sans dépendance
Optimisé	CNN Regularized	256 × 256	95,76 %	Excellent ratio Vitesse/Précision
ResNet50	<i>Transfer Learning</i> Fine-tuned	224 × 224	99,16 %	Performance quasi-parfaite
YOLOv8n	Détecteur <i>Zero-Shot</i> (COCO)	640 × 640	78,18 %	Localisation spatiale multi-objets

L'analyse de ces chiffres appelle une double lecture critique :

D'une part, sur le plan strictement statistique, YOLOv8n affiche la performance la plus faible de notre étude (environ 21 % de retard sur le Transfer Learning). Ce comportement est structurellement attendu : le modèle n'a subi aucun ajustement fin sur notre distribution d'images. De nombreuses erreurs et pénalités proviennent de la classe -1 : l'algorithme n'a pas échoué à distinguer le chat du chien, il a simplement manqué d'informations contextuelles ou de contraste pour isoler une boîte englobante valide.

D'autre part, d'un point de vue de l'ingénierie des données, YOLO apporte des fonctionnalités hors de portée de nos trois premiers CNN. Là où nos classificateurs s'effondreraient si une image contenait simultanément trois chiens et deux chats, YOLOv8 est capable de segmenter et de dénombrer les individus au sein d'une scène complexe.

Cette ouverture démontre que le choix d'un modèle ne se résume pas à maximiser l'*Accuracy* sur un jeu de données standardisé, mais doit être guidé par la nature de l'application cible : la classification pure pour du tri automatisé d'images isolées, et la détection d'objets pour de l'analyse vidéo ou de la robotique mobile en temps réel.

5. Analyse Globale des Résultats

5.1. Synthèse Métrique et Matrice Multi-Critères

Afin d'évaluer de manière holistique les différentes stratégies explorées au cours de ce projet, les modèles ont été confrontés sur le même jeu de test neutre et sanctuarisé (représentant 10 % du corpus initial). Cette comparaison ne se limite pas à l'exactitude globale (*Accuracy*), mais englobe la latence d'inférence (pertinente pour les applications en temps réel) ainsi que la nature conceptuelle de l'architecture.

Modèle / Architecture	Paradigme Algorithmique	Condition d'Entraînement	Temp. par image (s)	Test Accuracy
Expérience 1 : CNN Baseline	Classification binaire	<i>From Scratch</i> (Local)	0,0078 s	89,81 %
Expérience 2 : CNN Optimisé	Classification binaire	Régularisé (Local)	0,0072 s	95,76 %
Expérience 3 : ResNet50	Apprentissage par Transfert	<i>Fine-Tuning</i> hybride	0,0106 s	99,16 %
Expérience 4 : YOLOv8n	Détection / Régression	<i>Zero-Shot</i> (Poids COCO)	0,0142 s	78,18 %

5.2. Analyse Comparative des Comportements Inductifs

L'analyse croisée des métriques révèle des dynamiques d'apprentissage et des biais inductifs radicalement différents selon les choix d'ingénierie :

- L'apport de la régularisation locale (Baseline vs Optimisé) : La transition de la *Baseline* (89,81 %) au modèle optimisé (95,76 %) démontre la criticité de la maîtrise de la variance. Sans modification de la profondeur brute (5 blocs convolutifs), l'adjonction de la *Data Augmentation* et du *Dropout* a contraint le réseau à abandonner la mémorisation de primitives non discriminantes (bruits de fond, textures spécifiques du jeu de train) pour se focaliser sur des invariants morphologiques. La stabilisation des distributions d'activations par la *Batch Normalization* a quant à elle fluidifié la descente de gradient, optimisant la convergence sans surcoût computationnel mesurable à l'inférence.
- La supériorité des représentations généralistes (ResNet50) : Avec 99,16 % d'exactitude, le modèle ResNet50 frôle la perfection statistique. Ce saut quantique s'explique par la richesse sémantique des filtres convolutifs pré-entraînés sur ImageNet. Les couches basses de ResNet possèdent déjà une sensibilité optimale aux textures de pelages, formes d'oreilles et de museaux. L'ajustement des poids d'extraction s'avère presque superflu, validant l'hypothèse selon laquelle un grand modèle pré-entraîné surclassera systématiquement une architecture artisanale sur des tâches de vision standard, au prix d'une augmentation de la latence d'inférence de près de 47 % par rapport au CNN optimisé (0,0106 s contre 0,0072 s).
- La rupture fonctionnelle de la détection (YOLOv8n) : Le score de 78,18 % obtenu par YOLOv8n ne doit pas être interprété comme une défaillance algorithmique, mais comme la conséquence d'une divergence de paradigme. Évalué en mode *Zero-Shot* sans aucun réentraînement, YOLOv8 doit résoudre simultanément un problème de localisation (régression de coordonnées) et de classification multi-classes (80 classes COCO) sur une résolution spatiale plus élevée (640×640 pixels). Là où nos trois premiers CNN opèrent une classification forcée, YOLO s'accorde le droit de rejeter une image s'il ne valide aucune boîte englobante avec une confiance suffisante, générant des non-détections comptabilisées comme des erreurs statistiques (classe -1).

5.3. Analyse Technique des Erreurs et Limites de Généralisation

L'examen qualitatif des prédictions erronées met en évidence les limites physiques des modèles entraînés :

1. Bruits de bas niveau et occultations : Les erreurs persistantes du CNN optimisé se concentrent sur des images à faible contraste (sujets sombres sur fond nocturne) ou présentant des occultations partielles (animal caché derrière de la végétation). Dans

ces scénarios, les primitives géométriques locales (bords, angles) sont bruitées, ce qui induit en erreur le classificateur linéaire final.

2. Ambiguïtés inter-classes (Le problème de la distribution) : ResNet50, malgré sa robustesse, échoue sur de rares cas de plans serrés ou d'angles de vue atypiques (par exemple, une vue macro de gros plan sur le pelage ou la truffe). N'ayant pas accès à la géométrie globale de l'animal, le réseau sur-interprète des textures locales communes aux deux familles de quadrupèdes.
3. Analyse de la matrice de confusion YOLOv8 : La matrice de confusion de YOLOv8n met en lumière un phénomène d'asymétrie : le modèle montre une propension plus élevée à ignorer ou mal catégoriser certaines instances de chiens que de chats lorsque le sujet occupe l'intégralité de la matrice de pixels. L'absence d'un cadre entourant distinctement l'animal empêche l'algorithme d'ancrer correctement sa boîte englobante, provoquant une chute du score de confiance sous le seuil d'inférence minimal.

5.4. Arbitrage Industriel : Le Compromis Performance-Coût

En situation concrète de déploiement d'un système d'intelligence artificielle, le choix de l'architecture ne saurait se limiter à la contemplation des scores d'exactitude. Il relève d'un compromis technologique guidé par les contraintes matérielles (*Edge Computing* ou architectures embarquées) :

- Le CNN Optimisé représente la solution la plus efficiente si les ressources de calcul sont limitées (processeurs à basse consommation, absence de GPU dédié). Sa faible latence (0,0072 s) et son architecture compacte garantissent une empreinte mémoire minimale pour un taux de fiabilité très honorable (95,76 %).
- Le Transfer Learning (ResNet50) s'impose comme le standard industriel incontesté dès lors que la tolérance à l'erreur est nulle (applications de tri critique, diagnostic) et que l'infrastructure autorise un déploiement serveur ou cloud capable d'absorber la surcharge paramétrique du réseau.
- YOLOv8 transcende les limites de la classification. Si l'application cible exige d'analyser un flux vidéo continu contenant potentiellement plusieurs animaux simultanément, de suivre leurs déplacements ou de cartographier leur position spatiale, YOLO devient la seule option viable, ses capacités de localisation masquant largement son déficit initial d'exactitude en mode *Zero-Shot*.

6. Conclusion et Perspectives

Ce projet, mené dans le cadre de l'Unité de Valeur AC20, a permis de valider l'intégralité du cycle de conception, d'optimisation et d'évaluation critique d'un pipeline

d'apprentissage profond appliqué à la vision par ordinateur. À travers une démarche d'ingénierie incrémentale, nous avons apporté des réponses empiriques et théoriques à notre problématique centrale concernant les dynamiques de convergence et l'apport réel du *Transfer Learning* vis-à-vis d'une architecture conçue *From Scratch*.

Les conclusions majeures de cette campagne expérimentale se résument ainsi :

1. La fragilité structurelle des CNN basiques : L'implémentation de notre modèle *Baseline* a mis en évidence le fait qu'un réseau convolutif profond non régularisé souffre inévitablement d'un sur-apprentissage massif sur des données à forte variabilité. Bien qu'atteignant 89,81% d'exactitude, la divergence rapide des courbes de perte a matérialisé le risque de co-adaptation des neurones et de mémorisation du bruit de fond.
2. L'efficacité des techniques de régularisation : L'intégration conjointe de la *Data Augmentation*, du *Dropout* et de la *Batch Normalization* a prouvé qu'il est possible de contraindre le réseau à extraire des primitives hautement invariantes sans en modifier sa profondeur géométrique. Le bond de performance à 95,76% d'exactitude valide l'importance de la gestion du compromis biais-variance dans la modélisation de petits jeux de données.
3. L'état de l'art par le Transfer Learning : Le modèle ResNet50, fort de son pré-entraînement sur ImageNet et de ses connexions résiduelles, a démontré une hégémonie statistique absolue avec 99,16% de réussite. Cette performance montre que le transfert de connaissances depuis un domaine sémantique large est la stratégie à privilégier lorsque la tolérance à l'erreur est nulle, quitte à accepter une augmentation de 47% de la latence d'inférence par rapport à notre CNN optimisé.
4. L'élargissement fonctionnel de la détection : L'évaluation exploratoire de YOLOv8n a marqué une rupture conceptuelle majeure en substituant la classification globale forcée par une approche de localisation spatiale multi-objets. Si ses performances brutes en mode *Zero-Shot* (78,18) sont en deçà du *Transfer Learning* spécialisé, elles ouvrent la voie à des applications vidéo complexes en temps réel impossibles à traiter avec des classificateurs standards.

Perspectives de développement industriel :

Pour prolonger ce travail et l'inscrire dans une dynamique de production ou de déploiement réel sur des architectures matérielles embarquées (Edge Computing), trois axes technologiques complémentaires méritent d'être explorés :

- L'optimisation paramétrique par Quantification et Élagage (*Pruning & Quantization*) : Afin de déployer nos modèles les plus lourds (ResNet50) sur des microcontrôleurs ou des calculateurs embarqués à faibles ressources, il serait pertinent d'appliquer une quantification post-entraînement. Convertir les poids de la précision flottante 32

bits (\$FP32\$) vers des entiers 8 bits (\$INT8\$) permettrait de diviser par quatre l'empreinte mémoire et d'accélérer drastiquement la vitesse d'inférence, au prix d'une dégradation négligeable de l'exactitude.

- L'explicabilité des modèles par Grad-CAM (*Visual Explanations*) : Afin de lever le voile sur l'effet "boîte noire" de nos architectures, l'implémentation de l'algorithme Grad-CAM (Gradient-weighted Class Activation Mapping) constituerait un atout scientifique majeur. En calculant les gradients des scores de classe par rapport aux cartes de caractéristiques du dernier bloc convolutif, cette méthode permet de générer des cartes de chaleur visuelles. Nous pourrions ainsi vérifier rigoureusement si le réseau prend ses décisions en s'appuyant sur des critères morphologiques pertinents (oreilles, texture du pelage) ou s'il est biaisé par des éléments contextuels du décor (herbe, canapé).
- L'approche Data-Centric AI : Plutôt que de chercher à optimiser indéfiniment les hyperparamètres ou la profondeur des réseaux, la méthodologie contemporaine incite à améliorer la qualité intrinsèque des données. Une phase de filtrage avancé visant à équilibrer parfaitement les sous-représentations de certaines races au sein de notre dataset, couplée à des techniques d'augmentation synthétique via des réseaux antagonistes génératifs (GAN), permettrait de consolider la robustesse du modèle face à des cas réels hautement atypiques.

7. Bibliographie

Figure 1 : <https://www.advancinganalytics.co.uk/blog/2021/12/15/understanding-the-difference-between-ai-ml-and-dl-using-an-incredibly-simple-example>

Figure 2 et 3 : <https://deeplylearning.fr/cours-theoriques-deep-learning/fonctionnement-du-neurone-artificiel/>

Figure 4 : <https://www.datacamp.com/fr/tutorial/multilayer-perceptrons-in-machine-learning>

Figure 5 : <https://saturncloud.io/blog/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way/>

Figure 6 à 11 : Issue du projet

Lectures :

KHICHANE, Madjid. (2025). *Le Machine Learning et l'IA générative avec Python - De la théorie à la pratique (2e édition)*. Eni Editions

Géron, Aurélien. (2025). *Hands-On Machine Learning with Scikit-Learn and PyTorch*. O'REILLY